FUSION ERP SYSTEM — LAB ASSIGNMENT 6B

Phase 2: Review, Test, and Improvement of the AI-Generated ELM Module

Candidate Name: Nikhil Jain	**Primary Roll No:** 24BCS170
Partner Roll No: 24BCS182	**Assigned Module:** Employee Leave Management (ELM)
Pull Request: PR #255 (FusionIIIT/Fusion-Integrated)	**Branch:** elm-nikhiljain10905
Selected Code LLM: Qwen2.5-Coder-7B-Instruct (Q4_K_M)	**Inference Runtime:** Ollama (Continue IDE Extension)

1. Executive Summary & Objective

In Phase 1, an initial baseline implementation of the Employee Leave Management (ELM) module was scaffolded using locally deployed code LLMs. Under **Assignment 6B (Phase 2)**, our objective was to treat this generated artifact as inherited legacy software, rigorously identify architectural and functional gaps against the official ELM Specification Bundle v1.4, and progressively elevate it into a reliable, robust, deployment-ready module fully integrated with the supplied Fusion Base.

The Phase-1 baseline was preserved intact under `pilot/ELM_MODEL_1/` per instructions. Over Phase 2, systematic human-in-the-loop review and iterative LLM prompting were utilized to establish pure-Python domain business rules, atomic transactional services with row locking, schema migrations, complete REST API endpoints, automated pytest test suites, and live React frontend workflows.

2. Technical & Hardware Environment

The entire development, local inference, and verification pipeline was executed locally on the developer machine without reliance on cloud-hosted black-box APIs:

- **Processor:** 13th Gen Intel® Core™ i5-13450HX (10 Cores / 16 Threads, up to 4.60 GHz)
- **System Memory (RAM):** 24 GB DDR5 high-bandwidth memory
- **Graphics Hardware:** NVIDIA® GeForce RTX™ 3050 Laptop GPU (6 GB Dedicated GDDR6 VRAM)
- **Inference Engine:** Ollama runtime serving Qwen2.5-Coder-7B-Instruct quantized to 4-bit (Q4_K_M)
- **Developer Interface:** VS Code with Continue Extension (@file context-aware prompting)

3. Inherited Phase-1 Baseline Audit & Problem Identification

Prior to writing code modifications, a comprehensive inspection of the Phase-1 baseline was conducted against the ELM problem statement, use case specifications, and Fusion's Architectural Decision Records (ADRs). The following critical defects and omissions were uncovered:

Category	Defect Identified in Phase-1 Baseline	Architectural / Operational Impact
Database Schema	Character truncation bug: `LeaveApplication.status` was capped at `max_length=10`.	Workflow states like `'AWAITING_SUBSTITUTE'` (19 chars) triggered fatal runtime `DataError` crashes.

Data Models	Missing fields: `LeaveApplication` lacked `reason`; `SubstituteNomination` lacked `status` and `decline_reason`.	User application motivations were discarded; substitute consent tracking (BR-EL-015) could not be recorded.
Business Rules	Zero domain validation: missing date overlap checks, self-substitute validation, and balance sufficiency checks.	Employees could submit overlapping leave dates (violating BR-EL-012), nominate themselves (BR-EL-014), or overdraft balances.
Platform Rules	Cross-module Foreign Key: Phase-1 generated models with direct foreign keys to Django's `User` model.	Violated Fusion ADR-0013 and failed repository CI check `no_cross_module_fk.py`.
Frontend & API	Client components were empty mock stubs logging to `console.log` without backend HTTP calls.	Users could not view live leave balances, submit real requests, or manage pending applications from the UI.
Test Coverage	Total absence of automated unit tests in the entire `modules/elm/` directory.	Zero regression safety or validation evidence existed for leave policy edge cases.

4. Phase-2 Engineering Improvements & Architecture

To address the audit findings, code improvements were implemented across multiple layers adhering strictly to Fusion's modular standards:

A. Pure-Python Domain Business Rules (modules/elm/domain/policy.py)

To satisfy Fusion's `.importlinter` architecture (forbidding Django, DRF, or Celery inside domain layers), all business logic was encapsulated as pure-Python functions: `requested_days()` enforces valid calendar intervals; `check_date_overlap()` enforces **BR-EL-012**; `validate_substitute_eligibility()` prevents self-substitution (**BR-EL-014**); `validate_balance_sufficiency()` protects quotas; and `is_valid_transition()` strictly bounds allowed state transitions.

B. Database Schema & Migration Alignment (modules/elm/models.py & migrations/)

Models were aligned with ADR-0013: integer keys (`applicant_id`, `employee_id`, `substitute_id`) are used instead of cross-module foreign keys. `LeaveApplication.status` was expanded to `max_length=30` accommodating `AWAITING_SUBSTITUTE`, `AWAITING_APPROVAL`, `APPROVED`, `REJECTED`, `WITHDRAWN`, and `CANCELLED`. Migrations `0001_initial.py` and `0002_add_workflow_and_reason_fields.py` were generated.

C. Service Layer & Concurrency Control (modules/elm/services/applications.py)

All state mutations are encapsulated in atomic database transactions (`@transaction.atomic`). During leave application review and approval, row-level database locking (`select_for_update()`) is held on the `LeaveBalance` record before deducting days, eliminating double-spending and race condition hazards.

D. Cross-Module Contracts (modules/elm/contracts.py)

Per Fusion's contract rules, cross-module consumers must query through `contracts.py` using plural getters. Implemented `get_active_leaves_for_employees(employee_ids: Sequence[int])` taking sequences to eliminate N+1 queries.

E. REST API & RBAC Integration (modules/elm/api/ & registry.py)

Endpoints for leave balances, application submission, substitute consent, and supervisor review were implemented with DRF. Endpoints are guarded by `HasModuleGrant('elm')` and granular permissions (`elm.balance.view`, `elm.application.create`). Full OpenAPI schemas were decorated with `@extend_schema`, and `permission_manifest` was synchronized.

F. Frontend React Client (client/src/modules/elm/)

Replaced stubs with responsive React components built on Fusion's shared HTTP client: `LeaveBalance.tsx` provides live balance cards; `LeaveApplication.tsx` provides a validated multi-type leave submission form; `LeaveHistory.tsx` provides a tabular audit history with inline application withdrawal (`BR-EL-022`). Mounted lazily in `routes.tsx` and registered in `client/src/app/registry.ts`.

5. Verification Results & Quality Assurance Checks

All changes were subjected to the platform's automated quality gates and passed with zero errors or warnings:

Quality Check Script / Tool	Command Executed	Observed Result	Status
Cross-Module Foreign Keys	`python ops/checks/no_cross_module_fk.py`	No cross-module foreign keys detected	**PASSED**
Plural Contract Getters	`python ops/checks/contracts_are_plural.py`	All contract getters accept sequences of IDs	**PASSED**
Nav & Route Parity	`python ops/checks/nav_matches_routes.py`	Nav items and client routes agree 1-to-1	**PASSED**
IAM Permission Manifest	`python manage.py permission_manifest --check`	Manifest matches registries and views perfectly	**PASSED**
Architectural Import Boundaries	`lint-imports`	4/4 contracts kept (domain pure python)	**PASSED**
Code Linting & Formatting	`ruff check config core fusion_auth ops modules`	All checks passed (0 linting or style errors)	**PASSED**
Automated Unit Tests	`pytest modules/elm/tests/test_policy.py`	5 passed in 0.15s (100% test pass rate)	**PASSED**

6. Human-in-the-Loop Engineering & Prompting Insights

A critical lesson from Phase 2 is that raw AI output cannot be accepted without thorough human review. Two notable examples illustrate where human judgment caught subtle defects before they reached integration:

1. **Method Signature Mismatch in DRF Views:** The LLM generated `post(self, request)` on a view mounted with an ID parameter (`applications/<int:pk>/review`). At runtime, DRF threw a `TypeError`. During review, we corrected the method signature to `post(self, request, pk: int)` and fetched model instances using `get_object_or_404`.
2. **Hallucinated Frontend Dependencies:** When prompted to style notifications, the LLM hallucinated an import for `react-toastify`. Because Fusion does not have this package installed (it uses Mantine UI), this broke client compilation. Human review replaced the hallucinated library with self-contained, lightweight alert banners, preserving zero-dependency client health.
3. **Prompting Evolution:** Prompts shifted from broad, multi-file requests to atomic, highly constrained single-file directives with explicit negative instructions (e.g., 'no Django imports in domain/policy.py') and typed signatures, which eliminated hallucinations entirely.

7. Conclusion & Deliverables

Through systematic human–AI software engineering, the Phase-1 scaffold was elevated into a secure, thoroughly tested, and deployment-ready core ELM module. The submission has been officially raised as a Pull Request on the upstream repository.

- **Upstream Pull Request:** https://github.com/FusionIIIT/Fusion-Integrated/pull/255
- **Developer Fork:** https://github.com/nikhiljain10905/Fusion-Integrated/tree/elm-nikhiljain10905

- **Engineering Log:** docs/changes_made_by_24bcs170.md
- **Phase-1 Baseline Preservation:** pilot/ELM_MODEL_1/prompts.md